

Reflexion

Im Laufe der Implementierung des Clients mussten mehrere Aspekte des ursprünglichen Entwurfs verändert werden. Zu diesen Änderungen gehören unter anderem:

Datenstruktur für die Map: Im Klassendiagramm habe ich ursprünglich ein Set zum Speichern der Kartenfelder verwendet. Dies hat sich jedoch während der Implementierung geändert, da ich festgestellt habe, dass ich sehr häufig auf bestimmte Felder der Karte zugreifen muss, was mit einem Set ziemlich ineffizient ist und sich sogar negativ auf die Performance des Spiels auswirkt. Deshalb habe ich mich entschieden, stattdessen eine HashMap als Datenstruktur zu verwenden, um den Zugriff zu erleichtern und die Performance zu verbessern.

Kartengenerierung und -validierung: Da die Generierung und die Validierung der Karte eigentlich zwei verschiedene Aufgaben sind, habe ich sie bei der Implementierung in zwei getrennten Klassen untergebracht. Außerdem habe ich für jeden Aspekt der Kartengenerierung (z. B. Erzeugung von Gras, Wasser usw.) und der Validierung (z. B. Prüfung auf Inseln, Randfelder usw.) separate private Methoden eingeführt. Diese Modularisierung trägt dazu bei, dass das Prinzip der Single Responsibility besser eingehalten wird und macht den Code außerdem lesbarer und einfacher zu testen.

MVC-Implementierung: Ursprünglich hatte der Controller im Klassendiagramm mehrere Felder aus verschiedenen Model-Klassen. In der Implementierung habe ich das Pattern etwas anders umgesetzt, indem ich eine eigene Model-Klasse erstellt habe, die für die Speicherung von Daten und die Kommunikation mit der CLI über die firePropertyChange Methode verantwortlich ist, und der Controller hat nur noch das Model als Feld mit der Aufgabe, den Spielfluss über seine StartGame() Methode zu steuern.

Zusammenführung von Methoden in Player-Klasse: Eine weitere wesentliche Änderung war, dass ich die beiden Methoden findTreasure() und findFort() in der Player-Klasse zu einer einzigen Methode moveTowardsGoal() zusammengefasst habe, weil mir klar wurde, dass diese beiden Methoden im Wesentlichen die gleiche Funktionalität haben und dass ich die Redundanz im Code verringern konnte, indem ich die beiden Methoden einfach kombiniere und zusätzlich Fallunterscheidung einführe.

Y-Statements:

1. Im Kontext der Implementierung der Player-Klasse habe ich mich angesichts der Notwendigkeit, Redundanzen zu reduzieren, dafür entschieden, die beiden Methoden zur Suche des Schatzes und der Burg zu kombinieren, anstatt sie als separate Methoden anzulegen, um einen lesbareren und konziseren Code zu erhalten, unter Inkaufnahme, dass diese Änderung die Einführung zusätzlicher Logik in die Implementierung erfordert, um die Methode korrekt zu steuern.
2. Im Kontext der ClientMap habe ich mich angesichts der Notwendigkeit, häufig auf spezifische Felder in der Karte zuzugreifen, für die Verwendung einer HashMap-Datenstruktur entschieden und die Verwendung eines HashSets vernachlässigt, um den Zugriff auf die Felder zu erleichtern und die Performance des Spiels zu

verbessern, unter Inkaufnahme, dass diese Änderung ein Refactoring des Codes und eine Überarbeitung einiger der bestehenden Logik erfordert.

3. Im Kontext der Client HalfMap habe ich mich angesichts der Notwendigkeit, das Prinzip der Single Responsibility einzuhalten, für die Modularisierung der Kartengenerierung und -validierung in eigene Klassen entschieden und es vernachlässigt, sie beide als Teil der MapGenerator-Klasse zu implementieren, um sicherzustellen, dass jede Klasse nur eine Verantwortung hat, und auch um das Testen dieser beiden Aspekte zu erleichtern, unter Inkaufnahme, dass diese Änderung möglicherweise zu einer stärkeren Kopplung führt.